팹리스 실무형 일경험 프로그램

SDV 아키텍처 기반 자율주행 통합 시스템 구현 실습

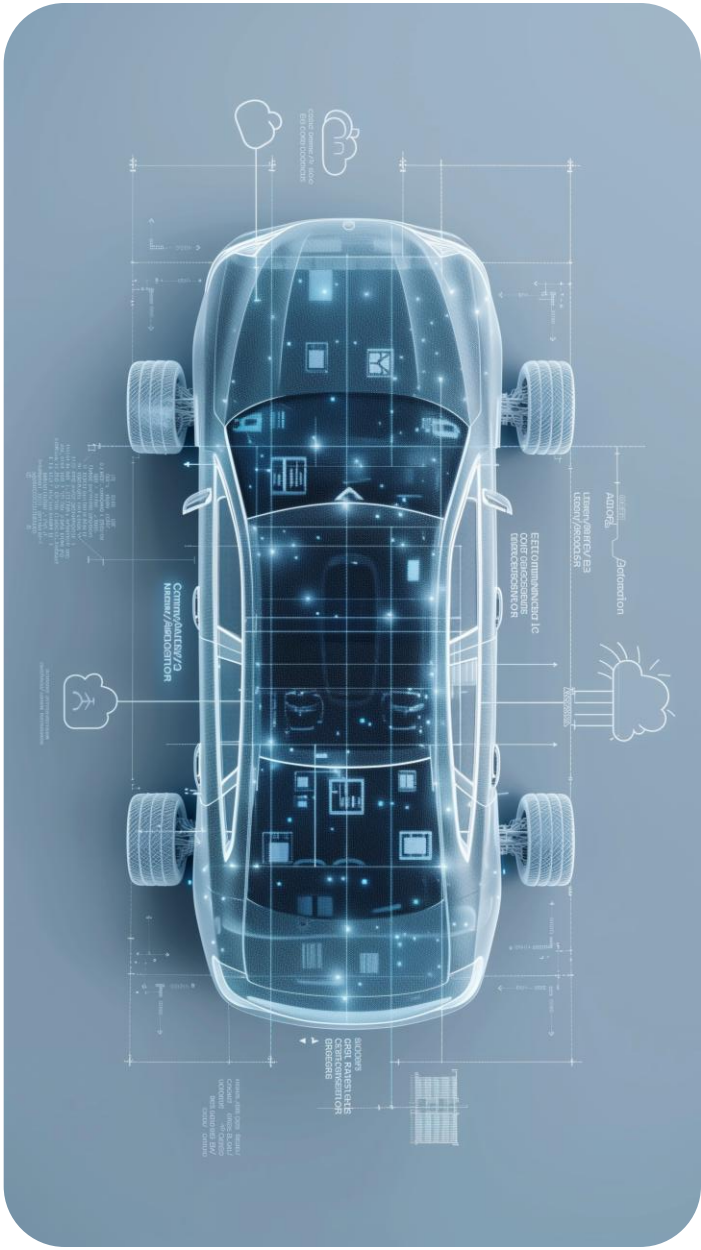
8일차

04 HPC Zone 구성

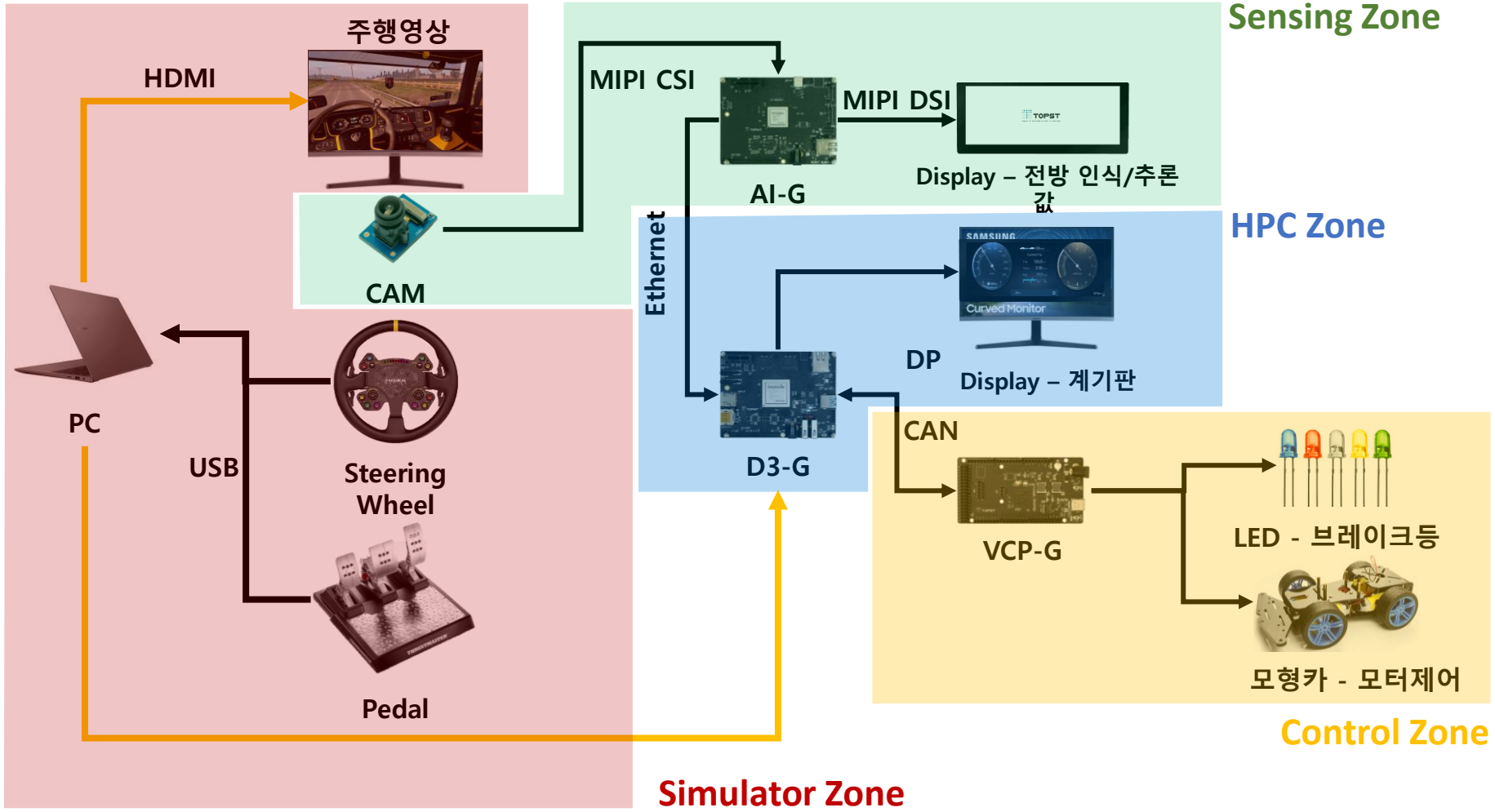
Telechips TOPST

목차

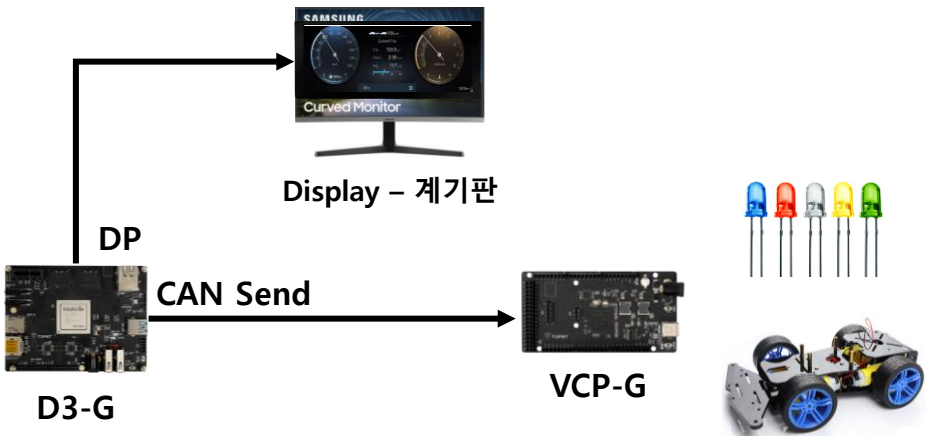
Table of Contents



Zonal 기반 차량 인터랙션 시스템-전체 시나리오



HPC Zone 이해



Zone Description

- 계기판 애플리케이션을 통해 시뮬레이터 차량 상태 확인
- D3-G 내 정의된 CAN 메시지 구성 및 송신

Task Goal

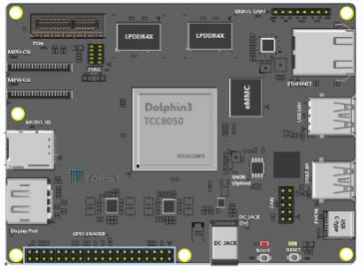
- Dashboard UI
- Dashboard Data Process
- CAN
- Handling CAN message on D3-G

Dashboard Data Process

ETS2-Telemetry

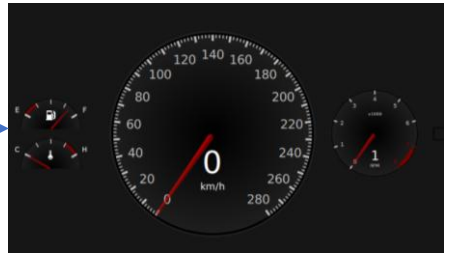
REST API

D3-G



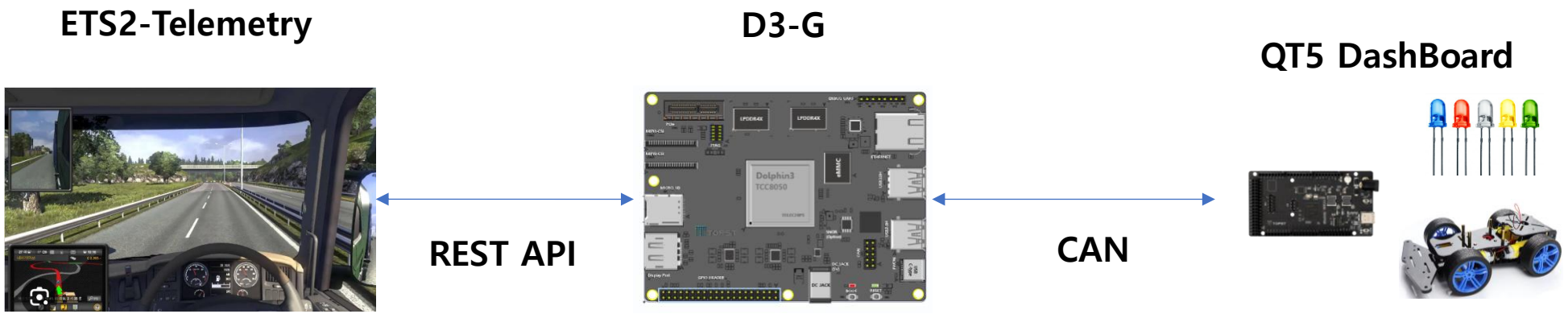
Socket

QT5 DashBoard



- Telemetry를 통해 차량의 상태 값을 실시간으로 받아 옴
- D3-G 에서 HTTP 프로토콜로 서버에 연결
- GET 메서드를 통해 값을 호출
- 차량 정보를 QT Dashborad에 전송하여 시각화

Can Data Process



- Telemetry를 통해 차량의 상태 값을 실시간으로 받아 옴
- D3-G의 A72에서 R5로 CAN 메시지 송신
- D3-G의 R5에서 VCP-G로 CAN 메시지 송신
- VCP-G에서 해당 CAN 메시지를 분석하여 센서(LED, LCD, 모터 등) 제어

Control App CAN 통신 작성

- 파일 수정 권한 부여
- $ cd Fabless_Education_blink/Step2
- $ sudo chmod 777 *

Control App CAN 통신 작성

- 편집기 실행
  - $ geany
- 왼쪽 상단의 File탭에 Open을 통해 Fabless_Education_blink/Step2의 Data_parsing.py 열기
- Can 통신을 위한 추가 기능 작성 (library)

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-


import json
import time
import socket
import threading
import http.client
import argparse
from urllib.parse import urlparse
from dataclasses import dataclass, field
from typing import Dict, Any, Optional


# ============================================================================
# External Library Imports (환경에 맞게 유지)
# ============================================================================
from Library.IPC_Library import IPC_SendPacketWithIPCHeader
```

Control App CAN 통신 작성

- **Can 통신을 위한 추가 기능 작성 (variables)**

```
# ================================================================================
# Configuration & Constants
# ================================================================================


class VCP_IO:
    """VCP IO Command Constants"""
    BREAK_LIGHT = 0x101
    TURN_SIGNAL = 0x102
    EMER_SIGNAL = 0x103
    HEAD_LIGHT  = 0x104
    FUEL_L      = 0x105
    MOTOR_A     = 0x106
    WHEEL       = 0x107

    ACTION_ON   = 0x01
    ACTION_OFF  = 0x02
    SUB_LEFT    = 0x01
    SUB_RIGHT   = 0x02
```

Control App CAN 통신 작성

- Can 통신을 위한 추가 기능 작성 (variables)

```
@dataclass
class Config:
    """Application Configuration"""
    IPC_PATH: str = "/dev/tcc_ipc_micom"
    TELEMETRY_URL: str = "http://192.168.137.1:25555/api/ets2/telemetry"

    # Frequencies (Hz)
    POLL_HZ: int = 60
    SEND_HZ: int = 60

    # Task Frequencies (Hz)
    TASK_HZ: Dict[str, int] = field(default_factory=lambda: {
        "break": 5, "head": 5, "turn": 2,
        "speed": 10, "fuel": 1, "wheel": 10
    })

    IDLE_RPM: int = 552
    BLINK_INTERVAL: float = 1


#
```

Control App CAN 통신 작성

- **Can 통신을 위한 추가 기능 작성 (variables)**

```
# ==============================================================================
# Main Monitor Class
# ==============================================================================


class ETSTelemetryMonitor:
    def __init__(self, config: Config):
        self.cfg = config
        self.running = threading.Event()
        self.lock = threading.Lock()

        # Resources
        self.ipc_file = None
        self.threads = []

        # State Data (Shared Resources)
        self.telemetry_data = {
            "rpm": 0.0, "speed_kmh": 0.0, "fuel_l": 0.0, "steer": 0.0,
            "park": False, "left_blinker": False, "right_blinker": False,
            "emergency": False, "headlights": False
        }

        # Previous State for Differential Updates
        self.prev_states = {}
```

Control App CAN 통신 작성

- **Can 통신을 위한 추가 기능 작성 (variables)**

```
def __enter__(self):
    """Resource Initialization"""
    try:
        print(f"[SYS] Opening IPC Device: {self.cfg.IPC_PATH}")
        self.ipc_file = open(self.cfg.IPC_PATH, 'wb')
    except FileNotFoundError:
        print(f"[ERR] IPC Device not found. Running in simulation mode.")
        self.ipc_file = None
    except Exception as e:
        print(f"[ERR] Failed to open IPC device: {e}")
        raise

    self.running.set()
    return self


def __exit__(self, exc_type, exc_val, exc_tb):
    """Resource Cleanup"""
    print("[SYS] Shutting down...")
    self.running.clear()

    for t in self.threads:
        t.join(timeout=1.0)

    if self.ipc_file:
        self.ipc_file.close()
    print("[SYS] Shutdown complete.")
```

Control App CAN 통신 작성

- Can 통신을 위한 추가 기능 작성 (variables)

```
# ------------------------------------------------------------------------
# Helper Methods
# ------------------------------------------------------------------------


def _send_ipc(self, io_type: int, value: int, subtype: Optional[int] = None) -> bool:
    """Thread-safe IPC Sender"""
    if not self.ipc_file:
        return False


    try:
        payload = bytes([subtype, value]) if subtype is not None else (value).to_bytes(1, 'big', signed=True)
        # Assuming IPC_Library handles file locking internally or is stateless enough
        IPC_SendPacketWithIPCHeader(self.ipc_file, 1, 0, io_type, payload)
        return True
    except Exception as e:
        print(f"[ERR] IPC Send Failed (0x{io_type:X}): {e}")
        return False
```

Control App CAN 통신 작성

- **Can 통신을 위한 추가 기능 작성 (variables)**

```
def _get_blink_state(self) -> bool:
    """Time-based blinker state calculator"""
    return (time.time() % (self.cfg.BLINK_INTERVAL * 2)) < self.cfg.BLINK_INTERVAL


def _parse_telemetry(self, js: Dict) -> Dict[str, Any]:
    """Safely parse JSON to internal state format"""
    def get_val(path, type_cast, default):
        cur = js
        try:
            for key in path.split("."):
                cur = cur[key]
            return type_cast(cur)
        except (KeyError, TypeError, ValueError):
            return default
    truck = "truck"
    left_on = get_val(f"{truck}.blinkerLeftOn", bool, False)
    right_on = get_val(f"{truck}.blinkerRightOn", bool, False)
    return {
        "rpm": get_val(f"{truck}.engineRpm", int, 0),
        "speed_kmh": get_val(f"{truck}.speed", int, 0),
        "fuel_l": get_val(f"{truck}.fuel", float, 0.0),
        "steer": get_val(f"{truck}.gameSteer", float, 0.0),
        "park": get_val(f"{truck}.parkBrakeOn", bool, False),
        "left_blinker": get_val(f"{truck}.blinkerLeftActive", bool, False),
        "right_blinker": get_val(f"{truck}.blinkerRightActive", bool, False),
        "emergency": (left_on and right_on),
        "headlights": get_val(f"{truck}.lightsBeamLowOn", bool, False),
    }
```

Control App CAN 통신 작성

- **Can 통신을 위한 추가 기능 작성 (variables)**

```
# ------------------------------------------------------------------------
# Logic Processors (Called by Workers)
# ------------------------------------------------------------------------


def _process_brake(self, state: bool):
    if self.prev_states.get("park") != state:
        action = VCP_IO.ACTION_ON if state else VCP_IO.ACTION_OFF
        if self._send_ipc(VCP_IO.BREAK_LIGHT, action):
            print(f"[IPC] Brake: {'ON' if state else 'OFF'}")
            self.prev_states["park"] = state


def _process_headlights(self, state: bool):
    if self.prev_states.get("headlights") != state:
        action = VCP_IO.ACTION_ON if state else VCP_IO.ACTION_OFF
        if self._send_ipc(VCP_IO.HEAD_LIGHT, action):
            print(f"[IPC] Headlights: {'ON' if state else 'OFF'}")
            self.prev_states["headlights"] = state


def _process_emergency(self, state: bool):
    if self.prev_states.get("emergency") != state:
        action = VCP_IO.ACTION_ON if state else VCP_IO.ACTION_OFF
        self._send_ipc(VCP_IO.EMER_SIGNAL, action)
        self.prev_states["emergency"] = state
```

Control App CAN 통신 작성

- **Can 통신을 위한 추가 기능 작성 (variables)**

```
def _handle_turn_signal(self, side_name: str, is_active: bool, subtype: int):
    """Generic handler for both left and right turn signals"""
    active_key = f"{side_name}_active"
    state_key = f"{side_name}_blink_state"

    # 1. Activation Change Check
    if self.prev_states.get(active_key) != is_active:
        self.prev_states[active_key] = is_active
        if not is_active:
            # Deactivated: Force OFF immediately
            self._send_ipc(VCP_IO.TURN_SIGNAL, VCP_IO.ACTION_OFF, subtype)
            self.prev_states[state_key] = False
            print(f"[IPC] {side_name.capitalize()} Signal: DEACTIVATED")
            return


    # 2. Blinking Logic (Only if active)
    if is_active:
        blink_on = self._get_blink_state()
        if self.prev_states.get(state_key) != blink_on:
            action = VCP_IO.ACTION_ON if blink_on else VCP_IO.ACTION_OFF
            if self._send_ipc(VCP_IO.TURN_SIGNAL, action, subtype):
                self.prev_states[state_key] = blink_on
```

Control App CAN 통신 작성

- **Can 통신을 위한 추가 기능 작성 (variables)**

```
def _process_speed(self, speed: float):
    # Only send if changed? Or continuously? preserving original logic (continuous)
    speed += 10
    self._send_ipc(VCP_IO.MOTOR_A, int(speed))


def _process_wheel(self, steer: float):
    # Map -1.0~1.0 to 0~127
    wheel_val = int((steer - 1.0) * 127 / -2.0)
    wheel_val = max(0, min(127, wheel_val))
    self._send_ipc(VCP_IO.WHEEL, wheel_val)
```

Control App CAN 통신 작성

- **Can 통신을 위한 추가 기능 작성 (variables)**

```
# ---------------------------------------------------------------------
# Worker Threads
# ---------------------------------------------------------------------


def _worker_http_poller(self):
    """Thread: Fetches data from ETS2 Telemetry Server"""
    u = urlparse(self.cfg.TELEMETRY_URL)
    path = u.path or "/"
    if u.query: path += "?" + u.query

    headers = {"User-Agent": "ETS2-Client/2.0", "Connection": "keep-alive"}
    period = 1.0 / self.cfg.POLL_HZ


    print(f"[NET] Poller started. Target: {u.hostname}:{u.port}")


    while self.running.is_set():
        start_time = time.monotonic()
        conn = None
        try:
            conn = http.client.HTTPConnection(u.hostname, u.port or 80, timeout=1)
            conn.request("GET", path, headers=headers)
            resp = conn.getresponse()
```

Control App CAN 통신 작성

- Can 통신을 위한 추가 기능 작성 (variables)

```
            if resp.status == 200:
                data = resp.read()
                js = json.loads(data.decode("utf-8", "ignore"))
                new_state = self._parse_telemetry(js)

                with self.lock:
                    self.telemetry_data.update(new_state)
            else:
                # Simple backoff on error
                time.sleep(0.5)


        except Exception as e:
            # print(f"[NET] Polling Error: {e}") # Verbose error suppression
            time.sleep(1.0)
        finally:
            if conn: conn.close()


        # Accurate Timing Loop
        elapsed = time.monotonic() - start_time
        sleep_time = period - elapsed
        if sleep_time > 0:
            time.sleep(sleep_time)
```

Control App CAN 통신 작성

- **Can 통신을 위한 추가 기능 작성 (variables)**

```
def _worker_can_sender(self):
    """Thread: Processes state and sends IPC commands"""
    print("[CAN] Sender thread started.")

    # Last execution time for each task
    last_runs = {k: 0.0 for k in self.cfg.TASK_HZ.keys()}

    while self.running.is_set():
        loop_start = time.monotonic()

        # Snapshot data to minimize lock time
        with self.lock:
            data = self.telemetry_data.copy()

        now = time.monotonic()
```

Control App CAN 통신 작성

- **Can 통신을 위한 추가 기능 작성 (variables)**

```
# 1. Brake
if (now - last_runs["break"]) >= (1.0 / self.cfg.TASK_HZ["break"]):
    self._process_brake(data["park"])
    last_runs["break"] = now


# 2. Turn Signals & Emergency
if (now - last_runs["turn"]) >= (1.0 / self.cfg.TASK_HZ["turn"]):
    if data["emergency"]:
        # Emergency overrides blinkers
        self._handle_turn_signal("left", False, VCP_IO.SUB_LEFT)
        self._handle_turn_signal("right", False, VCP_IO.SUB_RIGHT)
        self._process_emergency(True)
    else:
        self._process_emergency(False)
        self._handle_turn_signal("left", data["left_blinker"], VCP_IO.SUB_LEFT)
        self._handle_turn_signal("right", data["right_blinker"], VCP_IO.SUB_RIGHT)
    last_runs["turn"] = now


# 3. Headlights
if (now - last_runs["head"]) >= (1.0 / self.cfg.TASK_HZ["head"]):
    self._process_headlights(data["headlights"])
    last_runs["head"] = now
```

Control App CAN 통신 작성

- **Can 통신을 위한 추가 기능 작성 (variables)**

```
# 4. Speed
if (now - last_runs["speed"]) >= (1.0 / self.cfg.TASK_HZ["speed"]):
    self._process_speed(data["speed_kmh"])
    last_runs["speed"] = now


# 5. Fuel
if (now - last_runs["fuel"]) >= (1.0 / self.cfg.TASK_HZ["fuel"]):
    # Fuel logic simplified for IPC
    fuel_pct = int((data["fuel_l"] / 1500.0) * 100)
    fuel_pct = max(0, min(100, fuel_pct))
    self._send_ipc(VCP_IO.FUEL_L, fuel_pct)
    last_runs["fuel"] = now

# 6. Wheel
if (now - last_runs["wheel"]) >= (1.0 / self.cfg.TASK_HZ["wheel"]):
    self._process_wheel(data["steer"])
    last_runs["wheel"] = now


# Sleep briefly to prevent CPU hogging, but fast enough for highest Hz
time.sleep(0.001)
```

Control App CAN 통신 작성

- **Can 통신을 위한 추가 기능 작성 (variables)**

```
def start(self):
    """Start all worker threads"""
    workers = [
        self._worker_http_poller,
        self._worker_can_sender
    ]


    for func in workers:
        t = threading.Thread(target=func, daemon=True, name=func.__name__)
        t.start()
        self.threads.append(t)

    print("[SYS] All threads started.")

    # Main thread loop
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        print("\n[SYS] KeyboardInterrupt detected.")
```

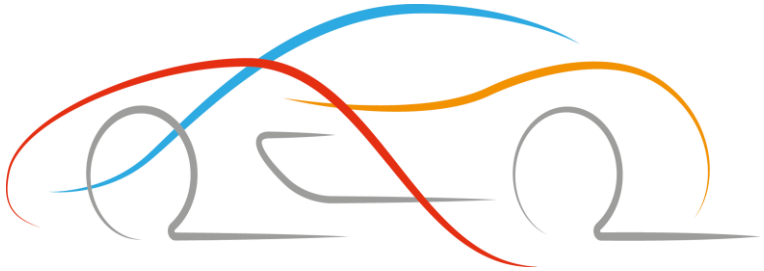
Control App CAN 통신 작성

- Can 통신을 위한 추가 기능 작성 (variables)

```
# ============================================================================
# Entry Point
# ============================================================================


if __name__ == "__main__":
    # Setup Argument Parser
    parser = argparse.ArgumentParser(description="ETS2 Telemetry to IPC Bridge")
    parser.add_argument("--ip", default="10.10.129.118", help="ETS2 Server IP")
    args = parser.parse_args()


    # Create Config
    config = Config()
    config.TELEMETRY_URL = f"http://{args.ip}:25555/api/ets2/telemetry"


    # Run Application
    with ETSTelemetryMonitor(config) as app:
        app.start()
```

Control App 실행 및 추가 기능 구현

- Can 통신을 위한 추가 기능 작성
  - Ctrl + s 로 저장
  - $ cd Fabless_Education_blink/Step2
  - $ sudo python3 Data_Parsing.py 로 실행
  - Vcp-g와 연동하여 동작하는지 확인



Thank You